

Object Oriented Programming (OOP)

In Python, classes and objects work hand in hand to organize and manage data. You build a class to define shared behavior, then create objects that use those behaviors.

In other words, a class is like a blueprint or template you use to create objects with.

Let's look at what classes are, and how to use them to create objects.

To create a class, you use the `class` keyword followed by the name of the class and a colon. Then within the class, you can add an initializer, along with any attributes and methods.

Attributes are like variables within a class, and are used to store data. Methods are functions defined within a class, and are the actions objects created with a class can perform.

Here's the basic syntax of a class:

```
class ClassName:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def sample_method(self):
        print(self.name.upper())
```

- `class ClassName` is made up of the `class` keyword to create a class, followed by the name of the class, here called `ClassName`. It is common in Python to use the **PascalCase** convention when naming classes.
- `def __init__(self, name, age)` is the special method automatically called when a new object is created. It initializes the attributes of the objects that will be created with the class.

In addition to that, the first parameter of `__init__` is always a reference to the specific object being created or used. By convention, this

parameter is named `self`, but technically, you can use any name. `self` lets you access the object's own attributes and methods.

- `self.name = name` and `self.age = age` are the attributes the objects will have.
- `def sample_method(self):` is the method each object created can call.
- `print(self.name.upper())` is what the `sample_method` method will do, in this case, it prints the name in uppercase.

If that all sounds like a lot, don't worry. Let's take a look at a similar example of a `Dog` class, and how you can create objects from that:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name.upper()} says woof woof!")
```

With this `Dog` class, you can create an object. Here's the basic syntax for creating objects from a class:

```
object_1 = ClassName(attribute_1, attribute_2)
object_2 = ClassName(attribute_1, attribute_2)
```

You can also call any of the methods defined in the class from each object:

```
object_1.method_name()
object_2.method_name()
```

Now let's create two dogs by using the `Dog` class as the blueprint:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
```

```
self.age = age

def bark(self):
    print(f"{self.name.upper()} says woof woof! I'm {self.age} years old!")

dog_1 = Dog("Jack", 3)
dog_2 = Dog("Thatcher", 5)

# Call the bark method
dog_1.bark() # JACK says woof woof! I'm 3 years old!
dog_2.bark() # THATCHER says woof woof! I'm 5 years old!
```

As you can see, we create two dog objects using the `Dog` class. When initializing `dog_1`, the string `Jack` and the number `3` are passed, which sets the `name` and `age` attributes for that instance. And `dog_2` is initialized with the string `Thatcher` and number `5` as its `name` and `age`, respectively.

Then when you call the `.bark()` method on `dog_1` and `dog_2`, you can see how both outputs differ, and use the unique `name` and `age` attributes you passed in when creating each object.

In summary, the difference between a class and an object is that a class is the template or the blueprint, and an object is what is created using that template.

Also, a class defines what data and behavior the object should have, and an object holds the actual data and uses that behavior. You write a class once, and you can make many objects from it, each with different data.

Methods and Attributes

In the last lesson, you learned about classes and how they act as blueprints for creating objects.

Here, we will dive deeper into attributes and methods.

Let's take a closer look at attributes first, then methods.

Attributes are variables that belong to an object, so they hold data. There are two kinds of attributes: instance attributes and class attributes.

Instance attributes are unique to each object created from a class, and you usually set them with the `__init__` method. Class attributes, on the other hand, belong to the class itself and are shared by all instances of that class.

To access an attribute, you use dot notation.

Here are examples of both instance and class attributes, and how to access them from objects:

```
class Dog:
    species = "French Bulldog" # Class attribute

    def __init__(self, name):
        self.name = name # Instance attribute

print(Dog.species) # French Bulldog

dog1 = Dog("Jack")
print(dog1.name)    # Jack
print(dog1.species) # French Bulldog

dog2 = Dog("Tom")
print(dog2.name)    # Tom
print(dog2.species) # French Bulldog
```

Note that you can access class attributes directly from the class itself, but you need to create an object and pass it data first before you can access instance attributes.

Cars are another good example, since all cars have a model and color:

```
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model

car_1 = Car("red", "Toyota Corolla")
car_2 = Car("green", "Lamborghini Revuelto")

print(car_1.model) # Toyota Corolla
print(car_2.model) # Lamborghini Revuelto

print(car_1.color) # red
print(car_2.color) # green
```

Methods are functions defined inside a class. With them, any object defined from a class can perform actions that operate on or modify its own data. You also access a method with dot notation.

For example, dogs can bark. So we can have a `bark` method in the `Dog` class like you saw in a previous lesson:

```
class Dog:
    species = "French Bulldog"

    def __init__(self, name):
        self.name = name

    def bark(self):
        return f"{self.name} says woof woof!"

jack = Dog("Jack")
jill = Dog("Jill")
```

```
print(jack.bark()) # Jack says woof woof!
print(jill.bark()) # Jill says woof woof!
```

A `Car` class can also have a `describe` method:

```
class Car:
    def __init__(self, color, model):
        self.color = color # Instance attribute
        self.model = model # Instance attribute

    def describe(self):
        return f"This car is a {self.color} {self.model}"

car_1 = Car("red", "Toyota Corolla")
car_2 = Car("green", "Lamborghini Revuelto")

print(car_1.describe()) # This car is a red Toyota Corolla
print(car_2.describe()) # This car is a green Lamborghini Revuelto
```

Special Methods and What Are They Used For?

Special methods in Python, also known as "magic methods" or "dunder methods", are special Python methods that start and end with double underscores (`__`). The word "dunder" itself comes from double underscores (**d** for double, **under** for underscores).

You've probably used special methods already without knowing it. Every time you write something like `3 + 4`, Python quietly runs `3.__add__(4)` under the hood. That's a special method in action. So while you *can* call special methods

directly, you rarely do. Something like `3 + 4` is much clearer and easier to read than calling `3.__add__(4)` yourself.

Apart from `__add__`, `__init__()` is another special method you'll see and use the most, as it's a class initializer. There are also others like `__len__()` and `__str__()`.

Think of special methods as the directors of the activities between a person programming and the Python language interpreter itself.

Remember, you don't need to call special methods directly. Instead, Python automatically calls them when certain actions happen. These operations include:

- **Arithmetic operations like addition, subtraction, multiplication, division, and others.** In addition, `__add__()` is called, `__sub__()` for subtraction, `__mul__()` for multiplication, and `__truediv__()` for division.
- **String operations like concatenation, repetition, formatting, and conversion to text.** `__add__()` is called for concatenation, `__mul__()` for repetition, `__format__()` for formatting, `__str__()` and `__repr__()` for text conversion, and so on.
- **Comparison operations like equality, less-than, greater-than, and others.** `__eq__()` is called for equality checks, `__lt__()` for less-than, `__gt__()` for greater-than, and so on.
- **Iteration operations like making an object iterable and advancing through items.** `__iter__()` is called to return an iterator and `__next__()` to fetch the next item.

Normally, Python data types like strings and numbers already know how to add things, do concatenation, compare for equality, be used in loops, and others.

But when you create your own class, Python won't know how to handle things automatically.

This is where special methods come in — they let you customize Python's built-in behavior.

Let's say you want to get the number of pages in book objects created with the class below, or compare them and get a readable string of the objects. Here's what happens without special methods:

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

book1 = Book("Built Wealth Like a Boss", 420)
book2 = Book("Be Your Own Start", 420)

print(len(book1)) # TypeError: object of type 'Book' has no len()
print(str(book1)) # <__main__.Book object at 0x102ed2900>
print(book1 == book2) # False even though they have the same number of pages
```

In the example:

- `len(book1)` failed because Python doesn't know how to get the length of your book object without `__len__()`
- `str(book1)` printed something like `<__main__.Book object at 0x102ed2900>` because that's the default representation when you don't use `__str__()`
- `book1 == book2` resulted in `False` because Python just checks if both objects are the same in memory, not by content.

Here's how you can define your own `__len__()`, `__str__()`, and `__eq__()` special methods to make working with objects created from the `Book` class easier:

```
class Book:
    def __init__(self, title, pages):
```



```

        self.title = title
        self.pages = pages

    def __len__(self):
        return self.pages

    def __str__(self):
        return f"'{self.title}' has {self.pages} pages"

    def __eq__(self, other):
        return self.pages == other.pages

book1 = Book("Built Wealth Like a Boss", 420)
book2 = Book("Be Your Own Start", 420)

print(len(book1)) # 420
print(len(book2)) # 420
print(str(book1)) # 'Built Wealth Like a Boss' has 420 pages
print(str(book2)) # 'Be Your Own Start' has 420 pages
print(book1 == book2) # True

```

Another example is a shopping cart where you do the following:

- Add items to the cart
- Remove items from the cart
- Get the number of items in the cart
- Check what items are in the cart
- Check if a specific item is in the cart
- Return or display an item at a specific index in the cart

While you might have a method that adds items to the cart and removes certain items from the cart, you can create special methods for all the other functionality:

- `__len__()` to get the length of the items in the cart
- `__iter__()` to loop through the items in the cart so you can see them
- `__contains__()` to check if a specific item is in the cart
- `__getitem__()` to return or display an item at a specific index in the cart

Here's an example of a `Cart` class with these user-defined methods and special methods:

```
class Cart:
    def __init__(self):
        self.items = []

    def add(self, item):
        self.items.append(item)

    def remove(self, item):
        if item in self.items:
            self.items.remove(item)
        else:
            print(f'{item} is not in cart')

    def list_items(self):
        return self.items

    def __len__(self):
        return len(self.items)

    def __getitem__(self, index):
        return self.items[index]
```

```
def __contains__(self, item):  
    return item in self.items
```

```
def __iter__(self):  
    return iter(self.items)
```

And here's how you can use them:

```
cart = Cart()  
cart.add('Laptop')  
cart.add('Wireless mouse')  
cart.add('Ergo keyboard')  
cart.add('Monitor')  
  
for item in cart:  
    print(item, end=' ') # Laptop Wireless mouse Ergo keyboard  
Monitor  
  
print(len(cart)) # 4  
print(cart[3]) # Monitor  
  
print('Monitor' in cart) # True  
print('banana' in cart) # False  
  
cart.remove('Ergo keyboard')  
  
print(cart.list_items()) # ['Laptop', 'Wireless mouse',  
'Monitor']  
  
cart.remove('banana') # banana is not in cart
```

And those are a few ways you'll use special methods in Python in the real-world.

Handle Object Attributes Dynamically

In a previous lesson, you learned about attributes being the variables that belong to an object. That means they hold data that describes the state or behavior of the object.

For example, a car would normally have a brand and model. The brand and model could make attributes for a `Car` class:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

my_car = Car('Lamborghini', 'Gallardo')
print(my_car.brand) # Lamborghini
print(my_car.model) # Gallardo
```

But sometimes, you might not know which attributes you need until your program is running. Imagine you're writing a script that receives attribute names from a user or a configuration file. Those are not attributes you can hardcode ahead of time.

That's where handling attributes dynamically comes in. This way, you can access, modify, check, or even delete attributes using their names as variables, and not as fixed names in your code. This gives your program the flexibility to respond to different data or user input on the fly.

Python gives you four handy built-in functions to dynamically work with object attributes. They are `getattr()`, `setattr()`, `hasattr()`, and `delattr()`.

They let you access, create, check, and remove attributes using variable names. Let's take a look at each one in action.

`getattr()` makes it possible to read an attribute from an object when you don't know its name until runtime. If the attribute doesn't exist, it raises an `AttributeError`, unless you provide a default value.

To use it, you pass in the object, attribute name, and an optional default value:

```
getattr(object, attribute_name, default_value)
```

Here's an example:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

person = Person('John Doe', 30)

print(getattr(person, 'name')) # John Doe
print(getattr(person, 'age')) # 30
print(getattr(person, 'city', 'Milano')) # Milano
```

In the example above, `Milano` is a default value because `city` doesn't exist in the `Person` class.

As we said earlier, the real power of `getattr()` is apparent when the attribute name comes from a variable, such as from user input or some file.

In that case, you can't use the regular `object.attribute_name` syntax because the attribute name is not fixed.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
person = Person('John Doe', 30)
```

```
attr_name = input('Enter the attribute you want to see: ')\nprint(getattr(person, attr_name, 'Attribute not found'))
```

In this case, if the user types in `name`, they see `John Doe`, and if they type in `age`, they see `30`. And if they type something that doesn't exist in the class like `email`, they see `Attribute not found`.

This is exactly where dynamic attribute handling shines. It lets your code respond to input and data it hasn't seen before.

In addition, you might want to look through all the attributes an object has, not just the ones you already know. The built-in `dir()` function lets you do that. It returns a list of all attribute names on the object. Here's how to use it:

```
class Person:
```

```
    def __init__(self, name, age):
```

```
        self.name = name
```

```
        self.age = age
```

```
person = Person('John Doe', 30)
```

```
# Loop through all attributes of the person object with dir()\nfunction
```

```
for attr in dir(person):
```

```
    # Ignore dunder methods like __init__ or __str__ and\n    regular methods
```

```
    if not attr.startswith('__') and not\n    callable(getattr(person, attr)):
```

```
        value = getattr(person, attr)
```

```
        print(f'{attr}: {value}')
```

```
# Output
# age: 30
# name: John Doe
```

In the loop above, `callable()` is a built-in function that returns `True` if the object passed to it can be called like a function or method, and `False` otherwise. By checking `not callable(getattr(person, attr))`, the loop skips over methods and only prints the data attributes like `name` and `age`.

The `setattr()` function lets you create a new attribute or update an existing one dynamically. The syntax looks like this:

```
setattr(object, attribute_name, value)
```

Here's an example that sets config attributes based on data from some config or environment variable file:

```
class Configuration:
    pass

# Data loaded at runtime (like from a config or env file)
settings_data = {
    'server_url': 'https://api.example.com',
    'timeout_sec': 30,
    'max_retries': 5
}

config_obj = Configuration()

# Dynamically set attributes using dictionary keys and values
for attr_name, attr_value in settings_data.items():
    setattr(config_obj, attr_name, attr_value)
```

```
print(config_obj.server_url) # https://api.example.com
print(config_obj.timeout_sec) # 30
```

There is also `hasattr()`. Before you do something with an attribute or delete it, it's a good practice to check if it exists. That's what `hasattr()` lets you do. It checks if an attribute exists and returns `True` or `False` based on the result.

Here's the basic syntax:

```
hasattr(object, attribute_name)
```

And here's an example that dynamically checks for the existence of attributes in a `Product` class instance:

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

product_a = Product('T-Shirt', 25)

required_attributes = ['name', 'price', 'inventory_id']

for attr in required_attributes:
    if not hasattr(product_a, attr):
        print(f"ERROR: Product is missing the required attribute: '{attr}'")
    else:
        # Access the attributes dynamically once their existence is confirmed
        print(f'{attr}: {getattr(product_a, attr)}')
```



```
# Output:
# name: T-Shirt
# price: 25
# ERROR: Product is missing the required attribute:
'inventory_id'
```

The ERROR output occurred because `inventory_id` is missing from the `Product` class and its instance.

Lastly, `delattr()` lets you remove an attribute dynamically:

```
delattr(object, attribute_name)
```

For example, imagine an object has been fully processed, then you decide to clean up any sensitive or temporary attributes that might exist before saving the final version. After that, you can use `dir()` to loop through the remaining attributes:

```
class UserSession:
    def __init__(self, user_id, token):
        self.user_id = user_id
        self.auth_token = token # sensitive
        self.temp_counter = 0 # temporary

session = UserSession(101, 'a1b2c3d4e5')

# List of attributes to remove dynamically before "saving" the
session
attributes_to_clean = ['auth_token', 'temp_counter']

# Dynamically remove specified attributes
for attr in attributes_to_clean:
    if hasattr(session, attr):
```

```
delattr(session, attr)

print(f'Removed attribute: {attr}')

print('\nFinal attributes remaining:')

# Loop through the remaining attributes with dir()
for attr in dir(session):
    # Ignore dunder methods like __init__ or __str__ and
    # regular methods
    if not attr.startswith('__') and not
callable(getattr(session, attr)):
        print(f' - {attr}: {getattr(session, attr)}')

# Output:
# Removed attribute: auth_token
# Removed attribute: temp_counter

# Final attributes remaining:
# - user_id: 101
```

And that's how you can handle attributes dynamically!

```
class BankAccount:
    def __init__(self, account_holder: str, initial_balance: float = 0.0):
        self.holder = account_holder
        self.balance = initial_balance

    def deposit(self, amount: float):
        if amount > 0:
            self.balance += amount
            print(f"Deposited ${amount:.2f}. New Balance:
${self.balance:.2f}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount: float):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew ${amount:.2f}. New Balance:
${self.balance:.2f}")
        elif amount > self.balance:
            print("Insufficient funds.")
        else:
            print("Withdrawal amount must be positive.")

    def display_status(self):
        print(f"Account Holder: {self.holder} | Balance:
${self.balance:.2f}")

class Bank:
    def __init__(self, name: str):
        self.name = name
        self.accounts = {}

    def create_account(self, name: str, initial_deposit: float):
        if name in self.accounts:
            print("Account already exists.")
        else:
            new_account = BankAccount(name, initial_deposit)
            self.accounts[name] = new_account
            print(f"Account created successfully for {name}.")

    def get_account(self, name: str) -> BankAccount:
        return self.accounts.get(name, None)
```

```

# --- Demonstration Workflow ---
if __name__ == "__main__":
    # 1. Initialize the Bank object
    my_bank = Bank("City Central Bank")
    print(f"--- Welcome to {my_bank.name} ---")

    # 2. Create customer accounts (Objects interacting within an object)
    my_bank.create_account("Alice", 500.0)
    my_bank.create_account("Bob", 150.0)

    print("\n--- Testing Transactions ---")
    # 3. Retrieve an account object and perform methods
    alice_acc = my_bank.get_account("Alice")
    if alice_acc:
        alice_acc.display_status()
        alice_acc.deposit(200.0)
        alice_acc.withdraw(100.0)
        alice_acc.withdraw(700.0) # Intentional fail test
        alice_acc.display_status()

```